DAS 839 — NoSQL Systems

End Semester Project — Phase 2 Report

Multi-Pipeline ETL and Reporting Framework for Web Server Log Analytics

Dataset	NASA HTTP Web Server Logs (July–August 1995)
Total Records	3,461,607 Malformed: 6
Pipeline 1	MongoDB — Fully operational
Pipeline 2	Apache Pig (Local mode) — Fully operational
Pipeline 3	MapReduce (mrjob local) — Fully operational
Pipeline 4	Hive (HiveServer2) — Fully operational
Result Store	PostgreSQL 16

Group Members

Kothamasu Naga Venkata Aditya	IMT2023033
Abhijit D	IMT2023054
Vishnu Balla	IMT2023097
Praneeth M	IMT2023555

International Institute of Information Technology, Bangalore

github.com/Aditya-KNV/NoSQL_Project_Phase1_v2

May 2026

Contents

1 System Architecture

The system provides a unified command-line interface to execute the same ETL workload across four distinct execution backends. A shared log parser, common query definitions, timestamp-based batch loader, and a single PostgreSQL result schema ensure pipeline equivalence across all implementations.

ETL Flow:

`Raw NASA Logs → Shared Parser → Batch Loader → Selected Pipeline → PostgreSQL → Report`

1.1 Project Structure

```
nosql_etl_project/
  config.py                  # DB credentials, file paths, pipeline settings
  main.py                    # Entry point: pipeline selection menu
  requirements.txt
  sql/schema.sql             # PostgreSQL table definitions
  parser/log_parser.py       # Shared NASA log parser (all pipelines)
  loader/
    batch_loader.py          # Timestamp-based batch splitter (Phase 2 new)
    db_loader.py             # PostgreSQL result loader (shared)
  pipelines/
    mongodb_pipeline.py      # Pipeline 1: MongoDB aggregation ETL
    pig_pipeline.py          # Pipeline 2: Pig local mode
    mapreduce_pipeline.py    # Pipeline 3: Native MapReduce (mrjob)
    hive_pipeline.py         # Pipeline 4: Hive (HiveServer2)
  reporting/
    report.py                # Reads from PostgreSQL, prints report
```

1.2 Infrastructure

Component	Technology	Version	Role
Orchestration	Python	3.12	Parsing, batching, coordination
Pipeline 1	MongoDB	7.0	Aggregation pipeline ETL
Pipeline 2	Apache Pig	0.17	Pig Latin scripts (local mode)
Pipeline 3	MapReduce	mrjob 0.7	Native MR (local runner)
Pipeline 4	Apache Hive	4.0.1	HiveQL queries (HiveServer2)
Result Store	PostgreSQL	16	Reporting database

Table 1: Infrastructure components

1.3 Phase 2 Changes from Phase 1

File	Status	Change
`parser/log_parser.py`	Unchanged	No changes required
`loader/batch_loader.py`	New	Timestamp-based batch splitter
`loader/db_loader.py`	Extended	Added `save_batch_metadata()`; `batch_id` INT→TEXT
`sql/schema.sql`	Extended	Added `batch_metadata` table
`reporting/report.py`	Extended	Batch summary table + interactive batch selector
`pipelines/mongodb_pipeline.py`	Updated	Per-batch temp collection; correct Q1/Q2/Q3
`pipelines/pig_pipeline.py`	Updated	Local mode only; fixed `COUNT(DISTINCT)` syntax
`pipelines/mapreduce_pipeline.py`	Completed	Full implementation from Phase 1 scaffold
`pipelines/hive_pipeline.py`	New	New pipeline
`main.py`	Updated	4 pipelines; `--batch-id` flag
`config.py`	Extended	Added `PIG_HOME`, `HIVE_*`, `MR_TMP_DIR`

Table 2: Phase 2 file changes

2 Dataset and Parsing

Source	Internet Traffic Archive — NASA Kennedy Space Center HTTP logs
URL	https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html
Files	`NASA_access_log_Jul95` (205 MB) and `NASA_access_log_Aug95` (168 MB)
Records	3,461,607 total Malformed: 6
Format	CLF (Common Log Format) — one HTTP request per line

2.1 Raw Log Format

```
host - - [DD/Mon/YYYY:HH:MM:SS -ZONE] "METHOD /path HTTP/x.x" status bytes

Example:
199.72.81.55 - - [01/Jul/1995:00:00:01 -0400] "GET /history/apollo/ HTTP/1.0"
    200 6245
```

2.2 Extracted Fields and Cleaning Rules

Structured fields extracted by the shared parser (`parser/log_parser.py`):

- `host`, `timestamp`, `log_date`, `log_hour`
- `http_method`, `resource_path`, `protocol_version`
- `status_code`, `bytes_transferred`

Cleaning rules:

- Missing bytes field (`-`) is treated as 0
- Malformed lines are counted and reported — never silently dropped
- Timestamps parsed from `DD/Mon/YYYY:HH:MM:SS` format
- `log_date` and `log_hour` are pre-computed by the parser and available directly in all pipeline queries

3 Batching Strategy

3.1 Phase 2: Timestamp-Based Batching

Phase 1 used a fixed batch size of 500,000 records. Phase 2 switches to **timestamp-based batching**: each unique calendar day in the log files becomes one independent batch.

Implementation (`loader/batch_loader.py`):

1. All log files are read sequentially
2. Each valid record is grouped by its `log_date` field (already extracted by the parser as `"YYYY-MM-DD"`)
3. Records are sorted chronologically by date
4. One batch is yielded per unique calendar day
5. Batch IDs are assigned sequentially: `batch-001`, `batch-002`, ...

Result: 58 batches — 28 from July (July 1–28) and 30 from August (August 1–31, noting some dates missing from the logs). Each batch is completely independent — Q1, Q2, Q3 are executed on that day's records only with no cross-batch aggregation.

Batch size computation:

- `batch_size` = count of valid records for that calendar day
- `avg_batch_size` = rolling mean: $\frac{\sum_{i=1}^{n} \text{batch\_size}_i}{n}$ after each batch
- Across all 58 batches: average batch size = 59,682.9 records

3.2 Batching Summary

Pipeline	Runtime (s)	Batches	Records	Avg Batch Size
MongoDB	207.97	58	3,461,607	59,682.9
Pig (Local)	761.53	58	3,461,607	59,682.9
MapReduce	1,044.85	58	3,461,607	59,682.9
Hive	10,129.39	58	3,461,607	59,682.9

Table 3: Phase 2 execution summary (all four pipelines, 58 batches)

4 Mandatory Query Set

All three queries are executed independently on each batch. Results are stored per-batch, per-run in PostgreSQL.

4.1 Q1 — Daily Traffic Summary

For each `(log_date, status_code)` within the batch: total request count and total bytes transferred.

Schema: `log_date, status_code, request_count, total_bytes`

4.2 Q2 — Top 20 Requested Resources

Top 20 resource paths by request count, with total bytes and distinct host count.

Schema: `resource_path, request_count, total_bytes, distinct_host_count`

Note on distinct hosts: Distinct host count is computed per-batch scope in all Phase 2 pipelines. This is consistent across all four pipelines.

4.3 Q3 — Hourly Error Analysis

For each `(log_date, log_hour)` within the batch: error requests (status 400–599), total requests, error rate, distinct error hosts.

Schema: `log_date, log_hour, error_request_count, total_request_count, error_rate, distinct_error_hosts`

Implementation note: Q3 requires two passes in all pipelines. The first pass counts total requests per (date, hour); the second filters status 400–599 for error counts and distinct hosts. The error rate is computed by joining the two result sets in Python.

5 PostgreSQL Reporting Schema

5.1 Schema Design

Six tables are used. Separate result tables per query (rather than a unified table with a query identifier) were chosen for clarity and query performance.

`run_metadata` — one row per pipeline run

Column	Type	Description
`run_id`	TEXT (PK)	UUID generated per run
`pipeline`	TEXT	MongoDB / Pig / MapReduce / Hive
`started_at`	TIMESTAMP	Run start time (UTC)
`finished_at`	TIMESTAMP	Run end time (UTC)
`runtime_seconds`	FLOAT	Total wall-clock execution time
`total_records`	BIGINT	Total valid records processed
`total_batches`	INT	Number of batches
`batch_size`	INT	Configured batch size (0 for timestamp mode)
`avg_batch_size`	FLOAT	Actual average batch size
`malformed_count`	BIGINT	Malformed lines encountered

Table 4: `run_metadata` schema

`batch_metadata` — one row per batch per run (new in Phase 2)

Column	Type	Description
`run_id`	TEXT	Reference to `run_metadata`
`batch_id`	TEXT	e.g. `batch-001`
`batch_date`	DATE	Calendar date covered by this batch
`source_file`	TEXT	Source log file basename
`batch_size`	INT	Records in this batch
`avg_batch_size`	FLOAT	Rolling average up to this batch
`records_processed`	INT	Same as `batch_size`
`malformed_count`	INT	Malformed lines in this batch
`runtime_seconds`	FLOAT	Wall time for Q1+Q2+Q3 on this batch
`batch_start_ts`	TIMESTAMP	Earliest log timestamp in batch
`batch_end_ts`	TIMESTAMP	Latest log timestamp in batch

Table 5: `batch_metadata` schema

Result tables — `q1_daily_traffic`, `q2_top_resources`, `q3_hourly_errors`

All three share the tracking columns:

Column	Type	Description
id	SERIAL (PK)	Auto-increment primary key
run_id	TEXT	Reference to run_metadata
pipeline	TEXT	Pipeline name
batch_id	TEXT	Batch identifier
executed_at	TIMESTAMP	Execution time of this batch

Plus query-specific columns as defined in Section 4.

6 MongoDB Pipeline — Results

Run Metadata: Runtime: 207.97s Batches: 58 Avg Batch Size: 59,682.9 Malformed: 6

6.1 Pipeline Implementation

Per-batch flow:

1. Drop _batch_tmp collection
2. insert_many all batch records into _batch_tmp
3. Run three $group aggregation pipelines (Q1, Q2, Q3)
4. Drop _batch_tmp
5. Write results to PostgreSQL via db_loader

MongoDB is the fastest pipeline (207.97s) due to in-memory aggregation with no subprocess spawning overhead. Each batch takes 0.9–6.0 seconds depending on day size.

6.2 Q1 — Daily Traffic Summary (batch-058: 1995-08-31)

Log Date	Status Code	Request Count	Total Bytes
1995-08-31	200	74,857	1,426,722,969
1995-08-31	302	2,325	201,134
1995-08-31	304	12,413	0
1995-08-31	400	4	0
1995-08-31	404	526	0

Table 6: MongoDB Q1 results for batch-058 (1995-08-31)

6.3 Q2 — Top 20 Requested Resources (batch-058: 1995-08-31)

Resource Path	Requests	Total Bytes	Distinct Hosts
/images/NASA-logosmall.gif	8,366	3,605,382	3,524
/images/KSC-logosmall.gif	4,057	3,943,100	3,143
/htbin/cdt_main.pl	3,746	14,428,390	1,137
/images/MOSAIC-logosmall.gif	3,383	1,041,447	2,321
/images/USA-logosmall.gif	3,352	662,688	2,297
/images/WORLD-logosmall.gif	3,318	1,877,883	2,275
/images/ksclogo-medium.gif	3,168	15,885,128	2,109
/htbin/cdt_clock.pl	2,760	1,928,185	464
/shuttle/countdown/images/countclock.gif	2,484	14,021,988	977
/ksc.html	2,144	13,986,597	1,308
/shuttle/missions/sts-69/count69.gif	2,088	81,370,101	1,554
/shuttle/countdown/	1,870	7,440,160	1,459
/shuttle/countdown/liftoff.html	1,660	9,628,641	989
/history/apollo/images/apollo-logo1.gif	1,595	1,590,588	1,322
/	1,570	9,428,370	1,187
/shuttle/missions/sts-69/endeavour-pad-prelaunch-*.gif	1,549	64,012,144	919
/images/launch-logo.gif	1,541	2,262,873	1,284
/images/ksclogosmall.gif	1,381	4,434,700	1,147
/shuttle/missions/sts-69/sts-69-patch-small.gif	1,199	8,293,158	1,008
/shuttle/missions/sts-69/mission-sts-69.html	1,139	16,110,173	901

Table 7: MongoDB Q2 results for batch-058 (1995-08-31)

6.4 Q3 — Hourly Error Analysis (batch-058: 1995-08-31, sample)

Log Date	Hour	Error Reqs	Total Reqs	Error Rate	Distinct Hosts
1995-08-31	0	21	2,586	0.0081	14
1995-08-31	1	20	2,046	0.0098	11
1995-08-31	2	13	2,215	0.0059	7
1995-08-31	6	12	2,702	0.0044	9
1995-08-31	7	20	4,557	0.0044	15
1995-08-31	10	31	6,283	0.0049	16
1995-08-31	13	34	5,948	0.0057	15
1995-08-31	14	63	5,513	0.0114	24
1995-08-31	21	44	2,463	0.0179	15
1995-08-31	23	26	2,457	0.0106	8

Table 8: MongoDB Q3 results for batch-058 (1995-08-31), selected hours

7 Apache Pig Pipeline (Local Mode) — Results

Run Metadata: Runtime: 761.53s Batches: 58 Avg Batch Size: 59,682.9 Malformed: 6

7.1 Pipeline Implementation

Per-batch flow:

1. Write batch records as TSV (`host, status_code, resource_path, log_date, log_hour, bytes_transferred`)
2. Generate Pig Latin script dynamically
3. Execute `pig -x local query.pig` (four JVM launches per batch: Q1, Q2, totals pass for Q3, errors pass for Q3)
4. Parse `part-*` output files
5. Write results to PostgreSQL

Phase 2 fix: Pig 0.17 does not support `COUNT(DISTINCT field)` in a `FOREACH GENERATE` statement. The nested `DISTINCT` pattern is used instead:

```
grp = GROUP logs BY resource_path;
res = FOREACH grp {
    unique_hosts = DISTINCT logs.host;
    GENERATE group AS resource_path,
             COUNT(logs) AS request_count,
             SUM(logs.bytes_transferred) AS total_bytes,
             COUNT(unique_hosts) AS distinct_host_count;
}
```

Each batch takes approximately 12–13 seconds. The dominant cost is four JVM cold-starts per batch (3–8 seconds each), not computation time.

7.2 Q1 — Daily Traffic Summary (batch-058: 1995-08-31)

Results identical to MongoDB (pipeline equivalence confirmed):

Log Date	Status Code	Request Count	Total Bytes
1995-08-31	200	74,857	1,426,722,969
1995-08-31	302	2,325	201,134
1995-08-31	304	12,413	0
1995-08-31	400	4	0
1995-08-31	404	526	0

Table 9: Pig Q1 results for batch-058 (1995-08-31) — identical to MongoDB

7.3 Q2 — Top 20 Requested Resources (batch-058: 1995-08-31)

Results identical to MongoDB (pipeline equivalence confirmed). See Table 3 for values.

7.4 Q3 — Hourly Error Analysis (batch-058: 1995-08-31)

Results identical to MongoDB (pipeline equivalence confirmed). See Table 4 for values.

8 MapReduce Pipeline — Results

Run Metadata: Runtime: 1,044.85s Batches: 58 Avg Batch Size: 59,682.9 Malformed: 6

8.1 Pipeline Implementation

The Phase 1 MapReduce scaffold is fully completed in Phase 2 using **mrjob** in local runner mode (no Hadoop cluster required).

Four `MRJob` classes are defined:

Class	Steps	Description
`MRDailyTraffic`	1	Q1: emits `(log_date, status_code)` → `(count, bytes)`; combiner partial sums
`MRTopResources`	2	Q2: step 1 groups by path collecting hosts in a set; step 2 negation-sort for top-20
`MRHourlyTotals`	1	Q3a: counts all requests per `(log_date, log_hour)`
`MRHourlyErrors`	1	Q3b: filters status 400–599; counts errors and distinct hosts

Table 10: MapReduce MRJob classes

Per-batch flow:

1. Serialise batch records to `mr_tmp/batch.jsonl` (absolute path to handle spaces in directory names)
2. Run `MRDailyTraffic` for Q1
3. Run `MRTopResources` for Q2
4. Run `MRHourlyTotals` + `MRHourlyErrors` for Q3; join in Python to compute error rate
5. Write results to PostgreSQL

Key implementation note: Project-level imports (`loader`, `db_loader`) are placed inside the `run()` function, not at module level. mrjob copies the pipeline file to a temp directory and re-executes it as a subprocess for each mapper/reducer task. Module-level imports would fail in that context since the project package structure does not exist in the temp directory. Only `json`, `os`, and `mrjob` are imported at module level.

Each batch takes approximately 17–18 seconds. The overhead comes from four Python subprocess launches per batch rather than JVM starts.

8.2 Q1 — Daily Traffic Summary (batch-058: 1995-08-31)

Results identical to MongoDB and Pig (pipeline equivalence confirmed). See Table 2.

8.3 Q2 — Top 20 Requested Resources (batch-058: 1995-08-31)

Results identical to MongoDB and Pig. See Table 3.

8.4 Q3 — Hourly Error Analysis (batch-058: 1995-08-31)

Results identical to MongoDB and Pig. See Table 4.

9 Hive Pipeline — Results

Run Metadata: Runtime: 10,129.39s Batches: 58 Avg Batch Size: 59,682.9 Malformed: 6

9.1 Pipeline Implementation

The Hive pipeline connects to HiveServer2 via `pyhive` and executes HiveQL queries over YARN-managed MapReduce jobs.

Per-batch flow:

1. Write batch records as TSV to local `hive_tmp/batch_data/batch.tsv`
2. Push TSV to HDFS: `hdfs dfs -put -f batch.tsv /user/praneeth/hive_input/batch-NNN/`
3. `DROP TABLE IF EXISTS batch_tmp`
4. `CREATE TABLE batch_tmp` with tab-delimited TEXTFILE format and `dfs.replication=1`
5. `LOAD DATA INPATH` (from HDFS, not local) to populate the table
6. Execute Q1, Q2, Q3 HiveQL queries via YARN MapReduce
7. `DROP TABLE IF EXISTS batch_tmp`; delete HDFS input directory
8. Write results to PostgreSQL

Key implementation decisions and issues resolved:

- Table name must not begin with an underscore: `_batch_tmp` is rejected by Hive's parser; renamed to `batch_tmp`.
- `LOAD DATA LOCAL INPATH` through HiveServer2 places the file on the server's local filesystem but YARN MapReduce tasks read from HDFS, resulting in 0 mappers and `return code 2`. Fixed by uploading the TSV to HDFS first and using `LOAD DATA INPATH` (without `LOCAL`).
- `mapred-site.xml` was empty (no `mapreduce.framework.name` set), causing MR to default to local runner while Hive submitted jobs to YARN — containers received 0 splits. Fixed by adding `mapreduce.framework.name=yarn` and `mapreduce.application.classpath`.
- `yarn-site.xml` was missing the shuffle auxiliary service. YARN containers failed with `auxService:mapreduce_shuff` `does not exist`. Fixed by adding `yarn.nodemanager.aux-services=mapreduce_shuffle`.
- Single-node cluster requires `dfs.replication=1` in `hdfs-site.xml` to avoid under-replicated blocks that stall MR tasks.

HiveQL queries follow the same logic as the other pipelines. Q3 uses two separate `SELECT` statements (totals and errors) joined in Python to compute the error rate, identical to the MapReduce and Pig implementations.

Each batch takes approximately 150–240 seconds, dominated by YARN container startup overhead (one MR job per query, two MR jobs for Q3), HDFS staging, and MapReduce shuffle for each of the three queries.

9.2 Q1 — Daily Traffic Summary (batch-058: 1995-08-31)

Log Date	Status Code	Request Count	Total Bytes
1995-08-31	200	74,857	1,426,722,969
1995-08-31	302	2,325	201,134
1995-08-31	304	12,413	0
1995-08-31	400	4	0
1995-08-31	404	526	0

Table 11: Hive Q1 results for batch-058 (1995-08-31) — identical to MongoDB, Pig, MapReduce

9.3 Q2 — Top 20 Requested Resources (batch-058: 1995-08-31)

Results identical to MongoDB, Pig, and MapReduce (pipeline equivalence confirmed). See Table 3 for values.

9.4 Q3 — Hourly Error Analysis (batch-058: 1995-08-31)

Results identical to MongoDB, Pig, and MapReduce (pipeline equivalence confirmed). See Table 4 for values.

10 Pipeline Equivalence Verification

10.1 Q1 — All Four Pipelines (batch-058: 1995-08-31)

Date	Status	MongoDB	Pig	MapReduce	Hive	Match
1995-08-31	200	74,857	74,857	74,857	74,857	
1995-08-31	302	2,325	2,325	2,325	2,325	
1995-08-31	304	12,413	12,413	12,413	12,413	
1995-08-31	400	4	4	4	4	
1995-08-31	404	526	526	526	526	

Table 12: Q1 pipeline equivalence verification — all four pipelines

10.2 Q2 — Request Counts (batch-058: 1995-08-31, top 5)

Resource	MongoDB	Pig	MapReduce	Hive	Match
/images/NASA-logosmall.gif	8,366	8,366	8,366	8,366	
/images/KSC-logosmall.gif	4,057	4,057	4,057	4,057	
/htbin/cdt_main.pl	3,746	3,746	3,746	3,746	
/images/MOSAIC-logosmall.gif	3,383	3,383	3,383	3,383	
/images/USA-logosmall.gif	3,352	3,352	3,352	3,352	

Table 13: Q2 pipeline equivalence verification (request counts) — all four pipelines

10.3 Q3 — Hourly Error Analysis (batch-058: 1995-08-31, sample)

Date	Hr	Mongo	Pig	MR	Hive	Mongo Rate	Hive Rate	Match
1995-08-31	0	21	21	21	21	0.0081	0.0081	
1995-08-31	1	20	20	20	20	0.0098	0.0098	
1995-08-31	14	63	63	63	63	0.0114	0.0114	
1995-08-31	21	44	44	44	44	0.0179	0.0179	

Table 14: Q3 pipeline equivalence verification — all four pipelines

Q1, Q2, and Q3 results are **identical** across all four pipelines: MongoDB, Pig, MapReduce, and Hive. Full pipeline equivalence confirmed.

11 Comparative Analysis

Aspect	MongoDB	Pig Local	MapReduce	Hive
Runtime (s)	207.97	761.53	1,044.85	10,129.39
Per-batch avg (s)	∼3.6	∼13.1	∼18.0	∼174.6
Batches	58	58	58	58
Query style	JSON agg.	Pig Latin	MRJob	HiveQL
Execution model	In-memory	JVM subproc	Py subproc	YARN MR
Distributed	No	No	No	Yes (YARN)

Table 15: Pipeline comparison — all four pipelines

11.1 Key Observations

- **MongoDB is fastest** (207.97s) due to in-memory aggregation with no subprocess spawning overhead. Batch times range from 0.9s (small days) to 6.0s (large days).
- **Pig Local** (761.53s) launches four JVM processes per batch. JVM cold-start dominates: 3–8 seconds per launch × 4 launches × 58 batches. Actual computation is negligible compared to startup cost.
- **MapReduce** (1,044.85s) launches four Python subprocesses per batch via mrjob. Each subprocess start costs ∼4 seconds. MapReduce is slower than Pig despite using Python (lighter) subprocesses

because mrjob adds serialisation overhead for inter-step data.

- **Hive is slowest** (10,129.39s, ∼174.6s per batch) because every query is submitted as a full YARN MapReduce job. Each batch requires 5–6 MR job launches (Q1, Q2, two passes for Q3, plus internal Hive planning jobs), each incurring YARN container allocation, HDFS staging, and MapReduce shuffle overhead. This overhead is fixed per query regardless of batch size, making Hive disproportionately expensive for the moderate record counts in these daily batches. Hive's strength lies in very large datasets where computation cost dominates startup cost — the opposite of this workload's profile.
- **Malformed record count** is consistently 6 out of 3,461,607 across all four runs, confirming reproducible parsing.
- **Q2 distinct host counts** are per-batch scope in all Phase 2 pipelines. A host appearing on two different days is counted as distinct in each day's batch. This is consistent and correct within the per-batch isolation model.
- **The timestamp-based batching** produces variable-size batches (27,121 records on 1995-07-28 to 134,203 on 1995-07-13), reflecting genuine variation in daily web traffic during the period.

12 Conclusion

Phase 2 completes the multi-pipeline ETL framework with all four execution pipelines fully operational. The system processes 3,461,607 NASA HTTP log records across 58 timestamp-based batches, executing Q1, Q2, and Q3 independently per batch on each pipeline.

Phase 2 Achievements:

- Timestamp-based batching: 58 batches (one per calendar day), replacing fixed-size batching
- MapReduce pipeline completed from Phase 1 scaffold (mrjob local runner)
- Hive pipeline fully operational: HiveServer2 + YARN MapReduce, 58 batches, 10,129.39s total runtime, results verified equivalent to all other pipelines
- Pig pipeline fixed: `COUNT(DISTINCT)` syntax corrected for Pig 0.17
- Per-batch result isolation: Q1/Q2/Q3 stored independently per batch per run
- `batch_metadata` table added: per-batch date, size, runtime tracking
- Interactive batch selector in report: view any specific day's results
- Pipeline equivalence verified: Q1/Q2/Q3 identical across all four pipelines

GitHub: https://github.com/Aditya-KNV/NoSQL_Project_Phase1_v2